



## READING SHEET

# Layouts & Rendering Strategies

Module 3 · Frontend: React &amp; Next.js · Day 37 · Week 8

## PROGRAM

CodeOps · Full Stack

## WORK TYPE

Revision + mini-project

## EST. TIME

2–3 hours

**Tip** How to use this sheet

This is your take-away for Day 37. Two things follow from yesterday's folder-as-router idea: layouts that wrap a segment and survive navigation, and the decision every route makes about when its HTML is produced. Keep `npm run build` open while you read — the circle and lambda markers in its output are the fastest way to see whether a route is doing what you intended.

## Part 1 · Layouts

A layout is a shell wrapping a route segment and everything beneath it — the Outlet pattern from Day 31, declared by file name instead of by component. The **root layout** is mandatory because there is no `index.html` in a Next.js project: it is the only place the `html` and `body` tags exist, which is also why global CSS is imported there.

```
import "../globals.css";

export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body><Header />{children}<Footer /></body>
    </html>
  );
}
```

app/layout.js — required, wraps everything

The `children` prop is the Day 27 idea again: whichever `page.js` matched the URL arrives there, exactly as Outlet delivered it — without importing anything.

## Nesting

```
app/
  layout.js          header + footer, every page
  menu/
    layout.js        adds the category sidebar
    page.js           -> root > menu > page
    [id]/page.js      -> root > menu > page
    checkout/page.js -> root > page (no menu layout)
```

a layout per segment, composed automatically

Layouts wrap rather than replace: every layout from the root down to the matched segment is applied, outermost first. You never register them anywhere — put the file in the folder and every route below it is wrapped.

## Layouts persist across navigation

| NAVIGATING /menu → /menu/kitfo | WHAT HAPPENS                    |
|--------------------------------|---------------------------------|
| Root layout                    | Stays mounted, keeps its state  |
| Menu layout and its sidebar    | Stays mounted, scroll preserved |
| The page component             | Replaced with the new one       |
| An open dropdown in the header | Still open                      |

**Warning A layout never re-runs**

Because it does not re-render on navigation, a layout cannot read the current pathname on the server or react to the route changing — if you need that, use a client component inside it. The preserved state is normally the feature: it is why the cart survived navigation on Day 31, and here you get it for free at every level of the tree.

**template.js and route groups**

```
                                a layout that does remount, and shells without paths
app/menu/template.js           // fresh instance per navigation
                                // state resets, effects run again

app/(shop)/layout.js           app/(admin)/layout.js
app/(shop)/menu/page.js        -> /menu    (no "shop" in the URL)
```

Choose `template.js` only when you need the reset — a page-entry animation, or an effect that should run on every visit. Round-bracket folders do not appear in the URL, so they are how you give the shop and the admin area completely different shells while keeping clean top-level paths. Only one root layout may exist; groups let you vary everything below it.

**Part 2 · When Pages Are Built**

The question every Next.js page answers is *when* its HTML is produced. That one decision drives speed, freshness, cost and what a crawler sees.

| STRATEGY          | HTML BUILT             | GOOD FOR                              |
|-------------------|------------------------|---------------------------------------|
| Static (SSG)      | Once, at build time    | Marketing pages, docs, a fixed menu   |
| Incremental (ISR) | At build, then re-made | A menu that changes a few times a day |
| Dynamic (SSR)     | On every request       | Per-user prices, dashboards, search   |
| Client (CSR)      | In the browser         | Interactive panels behind a login     |

They are not exclusive — one application uses several at once, chosen per route, and Next.js decides for each route separately based on what that route actually does.

**Static is the default**

app/about/page.js – static without asking

```
export default async function AboutPage() {
  const info = await getRestaurantInfo();
  return <article>{info.story}</article>;
}
// built once, served from a file or a CDN edge
```

Next.js prerenders every route it can. You never opt *in* — you only opt out, by using something that genuinely depends on the request. A static page does no server work per visit, hits no database, and can be cached at an edge close to Addis. Nothing else you can do is faster.

### What forces a route to be dynamic

| USING THIS                | MAKES THE ROUTE DYNAMIC                 |
|---------------------------|-----------------------------------------|
| cookies()                 | The response depends on who is asking   |
| headers()                 | Same — it varies per request            |
| searchParams              | The query string is part of the request |
| fetch(... no-store)       | You asked for fresh data every time     |
| dynamic = "force-dynamic" | You said so explicitly                  |

#### Warning You opt out by accident

Reading a cookie deep inside a helper makes the entire route dynamic, often without you noticing. This is the most common reason a page you expected to be static appears with a lambda in the build output. Check that output after every change.

### Incremental regeneration

app/menu/page.js – static, refreshed hourly

```
export const revalidate = 3600; // seconds

export default async function MenuPage() {
  const dishes = await getDishes();
  return <DishList dishes={dishes} />;
}
```

The first visitor after the window expires still gets the cached page instantly, and Next.js regenerates it behind them. Nobody waits, and the menu is never more than an hour out of date. Choose the window honestly: how stale can this be before someone is misled? A menu can be an hour old; a price during a promotion cannot.

## Part 3 · Controlling the Strategy

Caching is decided in two places: at the individual **fetch** call, and at the **route segment**. One page can cache the restaurant details for a day, revalidate the menu hourly and skip the cache entirely for the cart — the strictest option present decides how the route as a whole is rendered.

```
three ways to fetch the same menu

// cached indefinitely – part of the static build
await fetch(url, { cache: "force-cache" });

// re-fetched at most once an hour
await fetch(url, { next: { revalidate: 3600 } });

// never cached – forces the route dynamic
await fetch(url, { cache: "no-store" });
```

**Tip Be explicit**

Whether an uncached fetch is cached by default has shifted across Next.js releases. State what you want rather than relying on the current default, and confirm it in the build output.

**Segment configuration**

| EXPORT FROM page OR layout | EFFECT                          |
|----------------------------|---------------------------------|
| revalidate = 3600          | Regenerate at most hourly       |
| revalidate = 0             | Always dynamic                  |
| dynamic = "force-static"   | Prerender, ignore dynamic reads |
| dynamic = "force-dynamic"  | Render on every request         |

Exported from a layout, the setting covers every route in that segment — the tidiest way to say "everything under checkout must be rendered fresh". For a single piece of data, prefer setting the option on the fetch, which keeps the decision beside the thing it describes.

**Pre-building dynamic routes**

```
app/menu/[id]/page.js – every dish, at build time

export async function generateStaticParams() {
  const dishes = await getDishes();
  return dishes.map((d) => ({ id: d.id }));
}
// forty dishes -> forty static HTML files
```

A bracketed segment does not have to mean rendering on demand. If you know the ids at build time, Next.js prerenders one page per dish and serves them like any other static file. A new dish added after the build is still rendered on request and then cached — use `dynamicParams = false` if you would rather it return a 404.

**Addis Eats, route by route**

| ROUTE      | STRATEGY          | WHY                                               |
|------------|-------------------|---------------------------------------------------|
| /          | Static            | The story and address never change between builds |
| /menu      | ISR, 1 hour       | Dishes change occasionally; speed matters most    |
| /menu/[id] | Static via params | Every dish is known at build time                 |
| /cart      | Client            | The person's own state, and private               |
| /checkout  | Dynamic           | Reads the session cookie and live pricing         |

Build the habit of asking two questions of every route: **does this depend on who is asking**, and **how stale may it be**? Those two answers pick the strategy every time — and if the answers are "nobody in particular" and "quite stale is fine", leave it static and do nothing at all.

## Part 4 · Streaming

Layouts and rendering strategy meet in the loading experience. Because the shell can be sent before the page is ready, a route can show its header and sidebar immediately and stream the slow part in behind them. The server does not hold the response until the slowest query finishes — it sends what it has and continues down the same connection.

| MOMENT        | WHAT THE PERSON SEES                     |
|---------------|------------------------------------------|
| Immediately   | Header, nav and sidebar, all interactive |
| While loading | A skeleton where the dishes will be      |
| On resolve    | The dishes, with no layout shift         |

That is what `loading.js` really is: not a spinner, but a marker for the boundary between what can be sent now and what has to wait.

### Suspense for finer control

```

                                stream one slow part, not the whole page
export default function MenuPage() {
  return (<>
    <CategoryBar />                                {/* instant */}
    <Suspense fallback=<DishSkeleton />>
      <DishList />                                  {/* slow query */}
    </Suspense>
  </>);
}
```

`loading.js` replaces the *entire* page while anything in it loads. A `Suspense` boundary inside the page lets the fast parts render immediately and holds back only the component that is waiting — the same `Suspense` you used for lazy components on Day 34, now marking a boundary in a streamed server response.

### The file hierarchy

```

                                app/menu/ – five files, five responsibilities
layout.js    the sidebar – persists across navigation
template.js  only if you need a remount each visit
loading.js   the skeleton while the segment loads
error.js     "use client" – catches a throw in the segment
page.js      the screen itself
```

#### Warning A layout sits outside its own error boundary

`Next.js` composes these into layout wrapping error wrapping `Suspense` wrapping the page — the stack you wrote by hand on Day 34, now produced by five file names. But `error.js` catches errors in the page, not in the layout beside it. A layout that throws is caught by the boundary in the segment above.

## Assigned Reading

### Required

1. [nextjs.org — Layouts and templates](#) What / why: nesting, the root layout, and when a template is the right choice.
2. [nextjs.org — Route segment config](#) What / why: the reference for revalidate, dynamic and fetchCache — keep it open while you work.

### Optional — go deeper

- [generateStaticParams](#) — prebuilding dynamic segments, and what happens to ids you did not list.
- [Loading UI and streaming](#) — why the shell can be sent before the data is ready.

## Exercises

Work these in order and run `npm run build` after each one.

1. Build the root layout with header, footer and `globals.css`, and confirm it owns `html` and `body`.
2. Add `app/menu/layout.js` with a category sidebar and check it survives navigation to a dish.
3. Put a counter in the menu layout, navigate to a dish, and confirm the count is preserved.
4. Set `revalidate` on the menu route and confirm the build still marks it static.
5. Add `generateStaticParams` to the dish route and count the pages the build produces.
6. Add a `cookies()` read to the checkout and watch its marker change in the build output.
7. Wrap the dish list in `Suspense` so the sidebar renders before the dishes arrive.

## Mini-Project · Layouts and Strategies

### Tip What you build

Give Addis Eats a real shell and a considered rendering strategy for every route. A root layout with the header and footer, a nested menu layout whose sidebar survives navigation, static generation where it is safe, incremental regeneration where the data moves, dynamic rendering only where the request genuinely matters — and a streamed menu so the sidebar never waits for the dishes.

### Requirements

- A root layout owning `html` and `body`, importing global CSS, with header and footer.
- A nested menu layout with a sidebar, proven to keep state across a navigation.
- The menu route on a justified `revalidate` window, still marked static in the build.
- `generateStaticParams` on the dish route, producing one page per dish at build time.
- The checkout forced dynamic, with a comment naming the read that requires it.
- A `Suspense` boundary streaming the dish list behind an instantly rendered sidebar.
- `STRATEGY.md` listing every route, its strategy, and one line saying why.

### What to submit

Your GitHub repository with the layouts, the strategy settings and `STRATEGY.md`, plus the build output pasted into the `README`.



### Check yourself

- Does the build output mark each route the way your STRATEGY.md says it should?
- Does a counter in the menu layout keep its value when you open a dish?
- Did generateStaticParams produce one HTML file per dish?
- Can you name the exact line that makes the checkout dynamic?
- Does the sidebar appear before the dishes on a throttled connection?
- Can you justify every revalidate window in one sentence each?